山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202200130077	姓名：薛润泽	班级：人工智能
实验题目：实验六 栈		
实验学时：2	实验日期： 2023.10.25	

实验目的：

（1）掌握栈结构的定义与实现；

（2）掌握栈结构的使用。

软件开发工具：

DEV C++

1. 实验内容

题目描述：

创建栈类，采用数组描述；计算数学表达式的值。 输入数学表达式，输出表达式的计算结果。数学表达式由单个数字和运算符“+”、“-”、“”、“/”、“(”、“) ”构成，例如 2+3(4+5)-6/4。

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

数据结构：栈，结构体

整体思路描述：采用边输入边计算的方式，当输入‘)’时，计算‘(’与‘)’之间的内容，并把计算出的内容重新压入栈中，直到输入结束后，最终的栈中将不存在‘(’与‘)’，而将是一个算术表达式或者空栈。此时将栈中内容计算，并输出。

（1） struct Node{double data; char ope;}定义一个 Node 结构体，如果输入的是一个符号，则用 ope 来记录；如果输入的是一个数据或者是计算的‘(’，‘)’之间的内容，则用 data 来记录。

（2） class Stack; Stack 栈类，类中定义 element 数组用来记录栈中元素，listSize 用来记录栈中元素个数。Head 用来记录栈底元素，tail 用来记录栈顶元素。

（3） void pop();栈非空时弹出栈顶元素；

（4） bool empty();检测栈是否空

（5） T& top();返回栈顶元素

（6） double calc();用来计算一个括号内从右括号‘)’到左括号‘(’的值。因为除括号以外，所有的运算符都是二元运算符，所以计算的式子一定是数字、运算符、数字、运算符、……运算符、数字的形式。以一个数字开始，以一个数字结尾。定义 temp 用来记录栈顶数字，tempans 用来记录最终整个括号的答案。把加减号看错单项式的隔断，把以乘除号连接的数字看成一个单项式，使用 multidiv 用来暂存未计算完的单项式暂时结果，即暂存部分乘除法运算结果。将第一个数字记录到 temp 中并进行循环，每次处理一位符号和符号的下一个数字。如果该符号是‘+’或者‘-’则说明单项式结束。判断上一次进行的计算是否为乘除法运算，如果不是，直接将 temp 中数据加或者减到 tempans（记录一次括号中的最终答案）中；如果是乘除法运算，则将 temp 乘到 multidiv 中，再将 multidiv 加或者减到 tempans 中。如果该符号是‘*’或者‘/’号，那么将 temp 乘或者除到 multidiv 中。最后要单独将括号中的第一个数单独处理，如果第一个数右边的运算符为加减号，则将 temp 加减到 tempans 中（因为第一个数已经被记录到 temp 中）；如果

第一个数右边的运算符为乘除号，则将 temp 乘到 multidiv 中，并将 multidiv 加到 tempans 中。最后返回 tempans，即为一个括号内的运算值。

（7） void Solution();函数用来输入计算表达式，并完成动态括号内计算，最终输出整个表达式的运算结果。

3. 测试结果（测试输入，测试输出）

测试输入与输出：

```
3
1+6/1*7+2*1*4+9/1+2*0*9+9+7/(9*5)-1*6-0*8-7-9*2+6-(0-5-2*8-7-9*5*(6-5*5*2*6-2-7-5+6*7+6*9-1*0*0+3*0+2/1-6/6+5))
-9197.84
0-4-1/6*(1-(6/7)-4+6+2+6*1)-1*7+2-8*2+0-(4+6-6*1+(3-8*6/4-6-5)*6/4/8+7-1*4/9*5)-0/6+1-0-2+7-2+6*4-3*6+2/8+6+1*6*2
-3.47
5-3*9+5/1*5-9+1*8-6-8-4*1+5-2+9/3*2-2/5/(2-6)*2/7-9*0-2+4/6*6*7*8-8-8*6+8*9*(3+0*1/5/2*7*8+0-8*8-5+8/5*2-0)
-4362.57
```

4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

结果与预期相符

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

```
#include<iostream>

struct Node{
double data;//记录数据
char ope;//记录运算符,如果值为 0，则说明该点记录的是数据，否则是符号
};

template<typename T>
class Stack{
public:
    Stack(){
        element = new T[2005];//题目声明不超过 2000
        listSize = 0 ;
        head = 0;
        tail = 0;
    }
    ~Stack(){delete []element;}
    bool empty()const{return listSize == 0;}
    void push(T& a){
        tail++;
        element[tail] = a;
        listSize++;
    }
    void pop(){
        if(!empty()){
            tail--;
            listSize--;
        }
```

```
    }
    T& top(){
        return element[tail];
    }
    void clear(){//清空栈
        if(!this->empty()){
            this->pop();
        }
    }
    void OutStack(){
        while(!this->empty()){
            std::cout<<this->top();
            this->pop();
        }
    }
    double calc(){//计算一个括号内的值
        double temp = 0;//计算括号内运算结果,单项式结果
        double tempans = 0;
        double multidiv = 1;//特殊记录乘除法运算结果
        int sym = 0;//1  代表上一次操作为乘除法运算
        temp = this->top().data;
        this->pop();//最右侧数记录并弹出
        while( this->top().ope !='(' && !this->empty()){//当没有弹到'('则重复弹栈操作
            if(this->top().ope=='+'){
                if(sym == 1){//判断上一次的操作是否为乘除法操作
                    sym = 0;
                    multidiv *= temp;//如果上次操作为乘除法操作，那么说明 temp 中的数也是上次乘除法计算的一部分，为单项式的一部分，所以要将 temp 乘到 multidiv 中
                    tempans += multidiv; //因为这次的操作是加法操作，单项式结束，所以将单项式最后结果 multidiv 中的值加到最后的结果中
                    multidiv = 1;
                    this->pop();//弹出加号
                    temp = this->top().data;
                    this->pop();//弹出该数
                }
                else{
                    tempans += temp;//
                    this->pop();//弹出加号
                    temp = this->top().data;
                    this->pop();//弹出该数
                }
            }
            else if(this->top().ope=='-'){//同加法操作
                if(sym == 1){
                    sym = 0;
                    multidiv *= temp;
```

```
                    tempans -= multidiv;
                    multidiv = 1;
                    this->pop();
                    temp = this->top().data;
                    this->pop();//弹出该数
                }
                else{
                    tempans -= temp;
                    this->pop();//弹出减号
                    temp = this->top().data;
                    this->pop();//弹出该数
            }
            }
            else if(this->top().ope=='*'){//判断是否为乘法操作，如果是，则将 temp 中的值乘到 multidiv 中
                sym = 1;
                this->pop();//弹出乘号
                multidiv *= temp;
                temp = (this->top().data );
                this->pop();//弹出该数
            }
            else if(this->top().ope=='/'){//判断是否为除法操作，如果是，则将 temp 除到 multidiv 中
                sym = 1;
                this->pop();//弹出除号
                multidiv = 1.0 * multidiv / temp;
                temp = this->top().data;
                this->pop();//弹出该数
            }
        }
        //将括号内第一个单项式加入括号和中
        if(sym == 1){
            tempans += multidiv *temp;//括号内的第一个数单独进行处理
        }
        else{
            tempans += temp;
        }
        return tempans;
    }//calc 操作不弹出括号
    void Solution(){
        char c = getchar();
        while(c !='\n'){//当接收到的不是回车键时继续接受
            if(c == ')'){//当 c 是一个右边括号时
                double tempans = this->calc();
                Node temp;
                temp.ope = 0;
```

```
                    temp.data = tempans;
                    this->pop();//弹出左括号
                    this->push(temp);
                    c = getchar();
                    continue;
                }
                if(c >= '0' && c <= '9'){//说明压入的是一个数
                    Node temp;
                    temp.ope = 0;
                    temp.data = c -'0';
                    this->push(temp);
                }
                else{
                    Node temp;
                    temp.ope = c;
                    this->push(temp);
                }
                c = getchar();
            }
            double ans = this->calc();//最后栈中为一个不含括号的计算表达式，处理该表达式
            printf("%.2f\n",ans);
            return;
        }

private:
    T *element = nullptr;
    int listSize = 0;
    int head = 0;//头是栈的第一个元素索引，一般不进行操作
    int tail = 0;//尾是数组尾,随着元素压入、弹出栈而不断改变
};
int main(){
    Stack<Node> st;
    int n;
    std::cin>>n;
    char c = getchar();
    for(int i = 1; i<=n; i++){

        st.clear();
        st.Solution();
    }
    return 0;

}
```